PLIXORA

Product Requirements Document

Online Gaming Website | PERN Stack

Type	Stack	Platforms	Sign in
Online Gaming Site	**PERN**	**Web, Tablet, Mobile**	**Optional**

1. Project Overview

Plixora is a free online gaming website where people jump straight into browser games. No downloads, no installs, and no sign up. You open the page, pick a game, and you are playing within seconds, on a phone, a tablet, or a laptop. This document describes the whole product so a developer can build it end to end: the React website, the Express and Node API, the PostgreSQL database, and the admin panel the team uses to keep the catalogue fresh.

Field	Value
Product name	Plixora
Tagline	Play Instantly. Anywhere. Free.
Type	Online gaming website (instant play HTML5 portal)
Stack	PERN (PostgreSQL, Express, React, Node.js)
Architecture	Full stack: React frontend, Express API, PostgreSQL, admin panel
Primary colour	Plixora Purple #7C5CFF
Accent colour	Teal #00D1B2
Sign in	Optional, never required to play

2. Brand Guidelines

2.1 Colour Palette

Dark first, with a few vivid accents for a playful, modern feel. Keep body text at a minimum 4.5:1 contrast ratio.

Token	Name	Hex	Usage
Primary	Plixora Purple	#7C5CFF	Primary actions, links, active states
Secondary	Teal	#00D1B2	Accents, new badges, highlights
Tertiary	Pink	#FF5C9E	Hero gradient, promotional badges
Warning	Amber	#FFB84C	Ratings (stars), hot badges
Background	Deep Navy	#0F1226	App background
Surface	Panel Navy	#181C36	Cards, header, panels
Text	Off White	#ECEEF8	Primary text on dark surfaces
Muted	Cool Grey	#969CC4	Secondary text, placeholders

2.2 Mandatory Rules

- "Plixora" branding in header + footer on every page.

- Tagline is exactly "Play Instantly. Anywhere. Free." with no rewording.
- Use #7C5CFF for primary actions and active states. No colour substitutions.
- Keep it clean, no heavy ads, family friendly throughout.
- Keyboard accessible interactive elements. Alt text on all images.

3. Pages (Information Architecture)

These are the routes that make up the site. Sections 4 and 5 show how they look and what each one needs.

Page	Path	Purpose
Homepage	/	Featured, popular, new, and category rows
Browse	/category/:slug	Filterable, paginated grid for one category
Play	/game/:slug	Embedded game, full screen, details, related games
Search	/search?q=	Live suggestions and results grid
Sign in / Register	/login	Optional account access
Favourites / History	/favourites, /history	Saved and recently played games
Static	/about, /contact, /privacy, /terms	Information pages
Admin	/admin	Manage the catalogue (staff only)

4. Reference Screens

These are the approved mockups. Build each route to match the layout, spacing, and styling shown here.

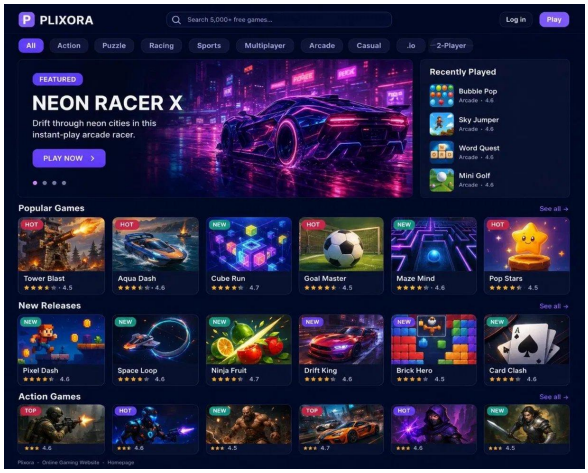

Figure 1. Homepage

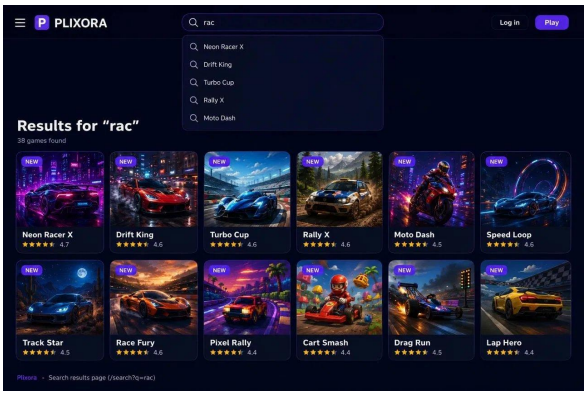

Figure 2. Search

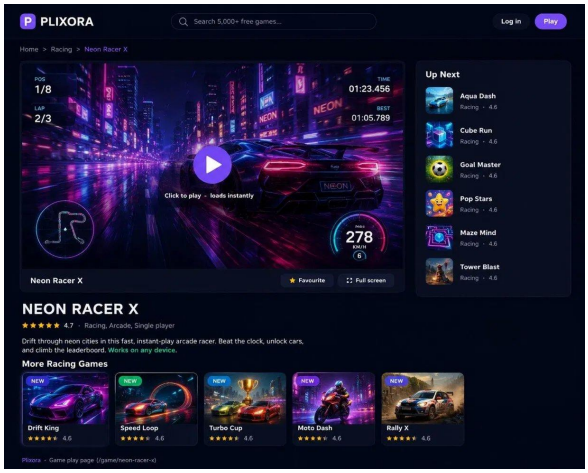

Figure 3. Play

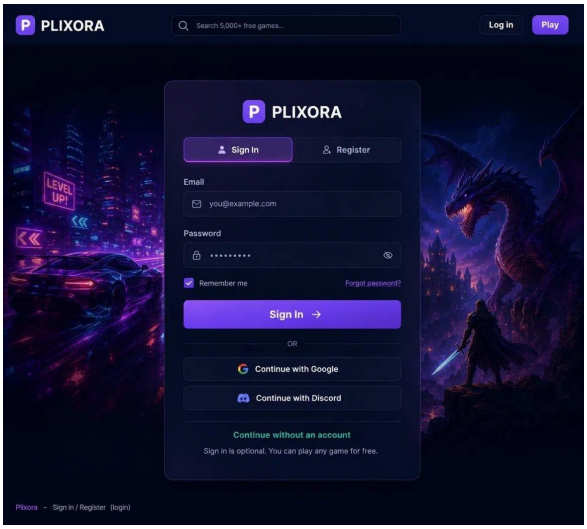

Figure 4. Sign in / Register

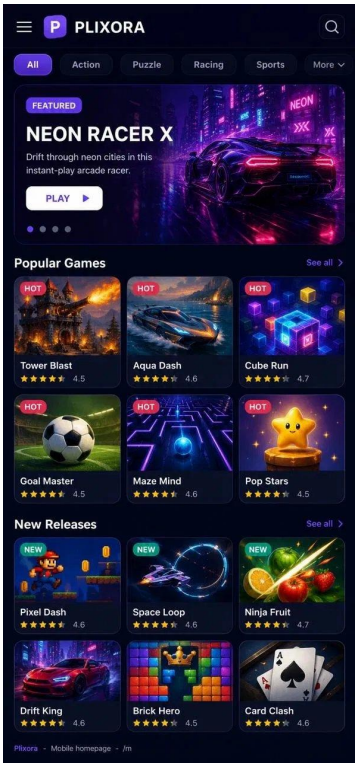

Figure 5. Mobile homepage

5. Frontend Requirements

The frontend is a React single page app using React Router. Build the homepage in this order:

1. Sticky header: logo, centred search bar, Log in and Play.
2. Scrollable category chips below the header.
3. Featured hero that rotates through 3 to 5 games.
4. Game rows: Popular, New, and one per category.
5. Reusable game card: thumbnail, title, badge, rating, hover play overlay.
6. Footer linking to About, Contact, Privacy, and Terms.

Responsive behaviour

Breakpoint	Cards per row	Navigation
1200px and up	5 to 6	Full category bar
768px to 1199px	3 to 4	Scrollable chips
Below 768px	2	Hamburger menu

6. Backend REST API

The React app and the admin panel both talk to this Express API. List endpoints support paging, filtering, and sorting; all inputs are validated; authentication is required only for account endpoints; and write and auth routes are rate limited.

Method	Endpoint	Purpose
GET	/api/games	List games (page, limit, category, sort)
GET	/api/games/:slug	Get one game
GET	/api/categories	List categories
GET	/api/search?q=	Search by title and tags
POST	/api/auth/register	Create an account
POST	/api/auth/login	Sign in, return token or session
POST	/api/auth/logout	End the session
GET / POST / DELETE	/api/favourites	List, add, remove favourites
GET / POST	/api/history	List or record recently played
POST	/api/ratings/:gameId	Submit a star rating

7. Database Schema (PostgreSQL)

Table	Key fields
games	id, slug, title, category_id, thumbnail_url, embed_url, rating_avg, plays, is_featured, is_published
categories	id, slug, name, cover_url
users	id, email, password_hash, display_name, created_at
favourites	id, user_id, game_id
play_history	id, user_id, game_id, played_at
ratings	id, user_id, game_id, stars

- Index games.category_id, games.title, and games.plays.
- Seed the database with at least 8 categories and 30 games.

8. Accounts and Persistence

- Sign in is optional and a visitor can play any game without an account.
- Signed out: favourites and recently played live in browser localStorage.
- Signed in: favourites and recently played sync to the account across devices.

- Passwords are hashed (for example bcrypt) and never stored in plain text.

9. Admin Panel

- Requires authentication. Anonymous access rejected.
- Create/edit/delete games and categories.
- Set thumbnails + embed URLs. Publish, unpublish, or feature games.
- Show basic stats: total plays and most popular games.

10. Non Functional Requirements

- Homepage loads under 2.5s on broadband. Catalogue/search API responses under 300ms.
- Layout works on desktop, tablet, mobile.
- Keyboard nav, focus states, alt text on images.
- Page titles + meta descriptions for SEO. HTTPS in production.

11. Technology Stack (PERN)

Layer	Technology
Frontend	React (Vite) with React Router
Backend	Node.js with Express
Database	PostgreSQL
Auth	JWT or express-session with bcrypt hashing
Game embed	HTML5 iframe with the Fullscreen API
Hosting	React static build, Express API host, managed PostgreSQL